PLAN DIRECTOR DE PROYECTO

Arquitectura Backend e Ingeniería de Software

Autor: David Nicolás Cardozo | Plataforma E-commerce (Indumentaria Urbana)

Este informe detalla la arquitectura técnica, la estructura de trabajo y el ciclo de vida de desarrollo para la construcción de una plataforma de comercio electrónico de alto rendimiento. El documento integra los fundamentos técnicos del *stack* de desarrollo con las mejores prácticas de la Ingeniería de Software, garantizando un sistema robusto, escalable y mantenible.

1. Arquitectura y Stack Tecnológico (Backend)

La base del sistema está construida sobre tecnologías modernas orientadas al rendimiento y la escalabilidad, aislando el entorno para garantizar la consistencia en cualquier etapa del desarrollo.

> **Node.js & Express.js:** Entorno de ejecución asíncrono y *framework* de enrutamiento modular para gestionar las peticiones HTTP de la API REST.

> **TypeScript:** Lenguaje de tipado estricto que previene errores lógicos y de tipeo en tiempo de desarrollo mediante el uso de interfaces, compilando luego a JavaScript puro.

> **PostgreSQL & Neon:** Base de datos relacional alojada en la nube bajo un modelo *serverless* para asegurar persistencia y alta disponibilidad.

> **Sequelize (ORM):** Capa de abstracción que mapea los objetos de TypeScript a sentencias SQL, agilizando la interacción con PostgreSQL sin depender de consultas crudas.

> **Docker:** Contenerización mediante `docker-compose` para aislar el servidor, la base de datos y los servicios auxiliares en una red interna.

> **Prettier & ESLint:** Herramientas de análisis estático y formateo para mantener la consistencia estética y la pureza del código fuente.

2. Estructura Lógica y Patrón de Diseño (MVC)

El sistema implementa una adaptación estricta del patrón Modelo-Vista-Controlador orientada a APIs REST, dividiendo las responsabilidades en capas aisladas para favorecer el mínimo acoplamiento y la máxima cohesión.

- **Capa de Enrutamiento (Routes):** Despachador central que intercepta la petición entrante, evalúa el verbo HTTP (GET, POST, PUT, DELETE) y la URL, derivando el tráfico al controlador correspondiente. Agrupado modularmente por recurso (usuarios, órdenes, inventario).
- **Capa de Intercepción (Middlewares):** "Aduana" de paso obligatorio. Filtra peticiones, parsea información y bloquea accesos no autorizados antes de que toquen la lógica de negocio.
- **Capa de Controladores (Controllers):** El cerebro del *backend*. Contienen las reglas de negocio, validan la operación, interactúan con los modelos mediante `async/await` y construyen la respuesta en formato JSON.
- **Capa de Datos (Models):** Representación lógica de las tablas de PostgreSQL. Mantienen la integridad referencial, exigencias de campos (NOT NULL, UNIQUE) y definen cardinalidades (1:1, 1:N, N:M con tablas pivote).

3. Seguridad y Control de Acceso

La protección de los *endpoints* se rige bajo un modelo estricto de autenticación por tokens y autorización basada en roles (RBAC).

- **Bcrypt & JWT (JSON Web Tokens):** Las contraseñas sufren un proceso de *hashing* criptográfico previo a su almacenamiento. Al autenticarse, el servidor emite un token firmado que actúa como llave de acceso temporal.
- **Escudos de Protección:** Implementación de `AuthMiddleware` para validar la presencia y firma del token en los *headers*, y `RoleMiddleware` para asegurar que el usuario posea los privilegios necesarios (ej. *admin* vs *cliente*).
- **Prevención de Inyecciones (Mass Assignment):** El controlador de registro público fuerza internamente el rol a 'cliente', ignorando cualquier intento de escalar privilegios desde el payload de la petición.
- **Gestión Administrativa Cerrada:** Los superusuarios y administradores se inicializan exclusivamente de forma interna mediante *Seeders* o acceso directo a PostgreSQL, cerrando toda brecha pública.

4. Motor Transaccional y Principios ACID

Para proteger la integridad financiera y el inventario del e-commerce, el backend ejecuta procesos críticos bajo el estándar ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).

Operaciones complejas como el **Checkout** se procesan en bloque (leer carrito, crear orden, registrar ítems, descontar stock). Si todas las validaciones son correctas, se ejecuta un `COMMIT`. Si cualquier regla falla (ej. stock insuficiente), el sistema lanza un `ROLLBACK` automático, abortando la transacción y manteniendo la base de datos en su estado prístino, previniendo cobros fantasma o inconsistencias.

5. Fases de la Ingeniería de Software

El ciclo de vida del proyecto no se basa en codificación impulsiva, sino en una arquitectura metódica dividida en cuatro fases estratégicas.

Fase 1: Ingeniería de Requerimientos (El Plano del Hardware)

- **Filtro W5HH:** Definición estricta de los objetivos del proyecto (Qué, Por qué, Quién, Cuándo, Dónde, Cómo, Cuánto).
- **Modelado de Comportamiento:** Elaboración de Casos de Uso (ej. Flujo de compra del cliente, gestión de inventario del administrador) identificando actores, disparadores y escenarios de excepción.
- **Diagrama de Entidad-Relación (DER):** Trazado visual de la estructura de la base de datos, mapeando las relaciones entre `Usuarios`, `Órdenes` y `Productos` para garantizar la normalización antes de escribir código.

Fase 2: Metodología y Gestión (El Sistema Operativo del Equipo)

- **Agilidad Práctica:** Enfoque iterativo, evolutivo e incremental. El desarrollo avanza mediante *sprints* cortos, priorizando entregas funcionales (ej. Incremento 1: Catálogo; Incremento 2: Carrito y Checkout).
- **Gestión de las 4 P:** Organización clara del Personal, fijación del Producto, estructuración del Proceso y control del Proyecto para evitar ambientes frenéticos.

Fase 3: Diseño y Arquitectura (Ensamblaje de Componentes)

- **Máxima Cohesión y Mínimo Acoplamiento:** Los módulos se diseñan con responsabilidades únicas e independientes (el módulo de pagos opera aislado del de inventario).

- **Auditoría FURPS:** La arquitectura se evalúa continuamente frente a sus atributos de calidad: Funcionalidad, Usabilidad, Confiabilidad, Rendimiento (Performance) y Mantenibilidad.

Fase 4: Testing y Control de Calidad (Benchmarking Final)

- **Pruebas Unitarias:** Aislamiento y validación de algoritmos críticos (ej. cálculo de impuestos y totales del carrito) en entornos controlados.
- **Pruebas de Integración y Regresión:** Verificación de las interfaces entre módulos (ej. conexión entre el *frontend* y los *endpoints* del *backend*). Uso de *Mocks* y *Stubs* para emular pasarelas de pago y servicios de terceros.
- **Pruebas de Humo (Smoke Testing):** Validaciones rápidas de despliegue para asegurar que los componentes vitales (conexión a DB, arranque del servidor) operen tras cada compilación.